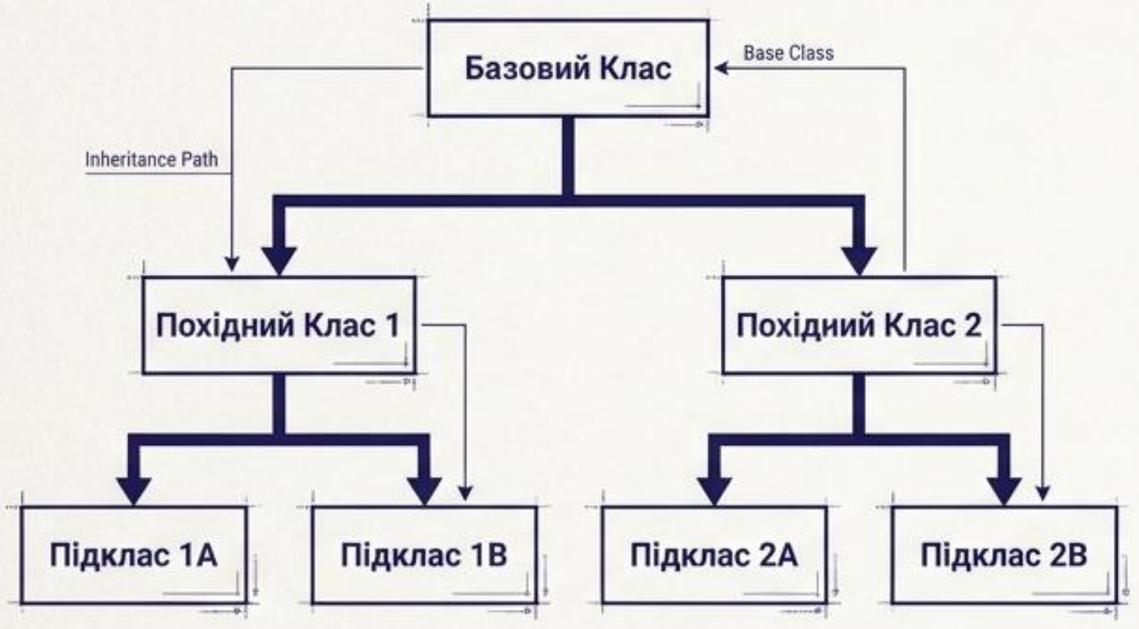
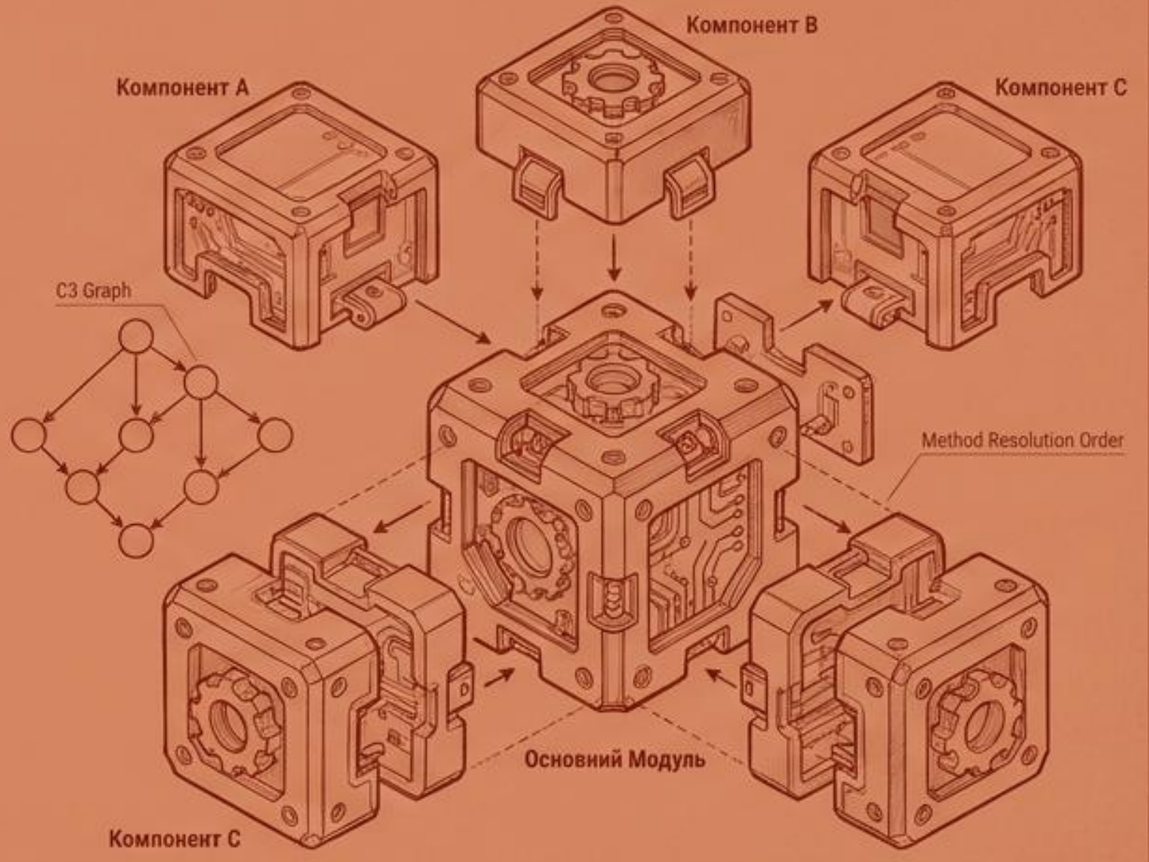
Архітектура ООП: Спадкування, Композиція та C3-Лінеаризація

Посібник архітектора з побудови масштабованих програмних систем





Gemini Notebook

Повторне використання коду

«Віддавайте перевагу композиції об'єктів над спадкуванням класів.»

— The Gang of Four (GoF),
Design Patterns (1994)

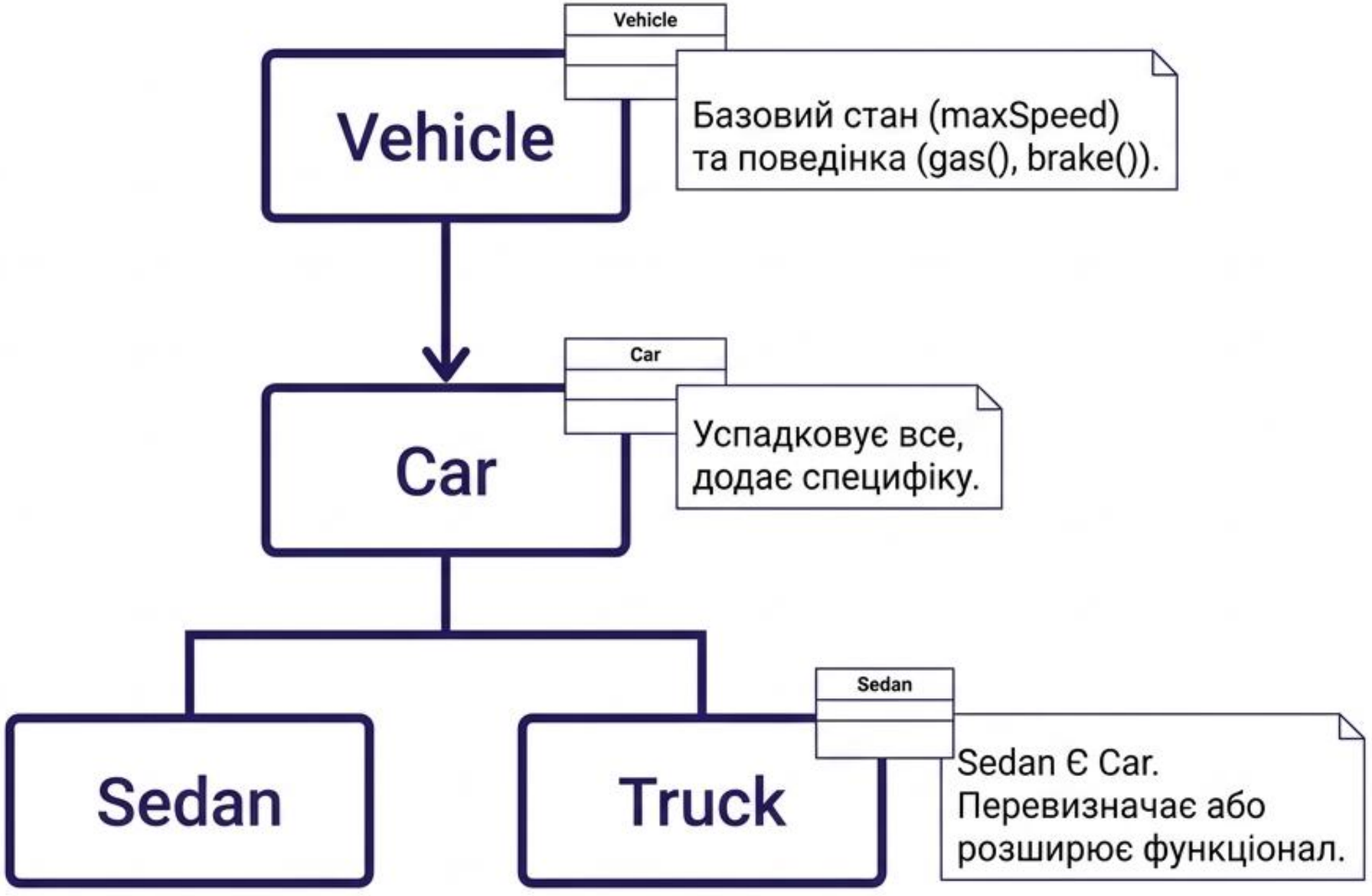
Шлях 1: Спадкування

Відношення «IS-A» (Є).
Жорстка таксономія.

Шлях 2: Композиція

Відношення «HAS-A» (Має).
Гнучка модульність.

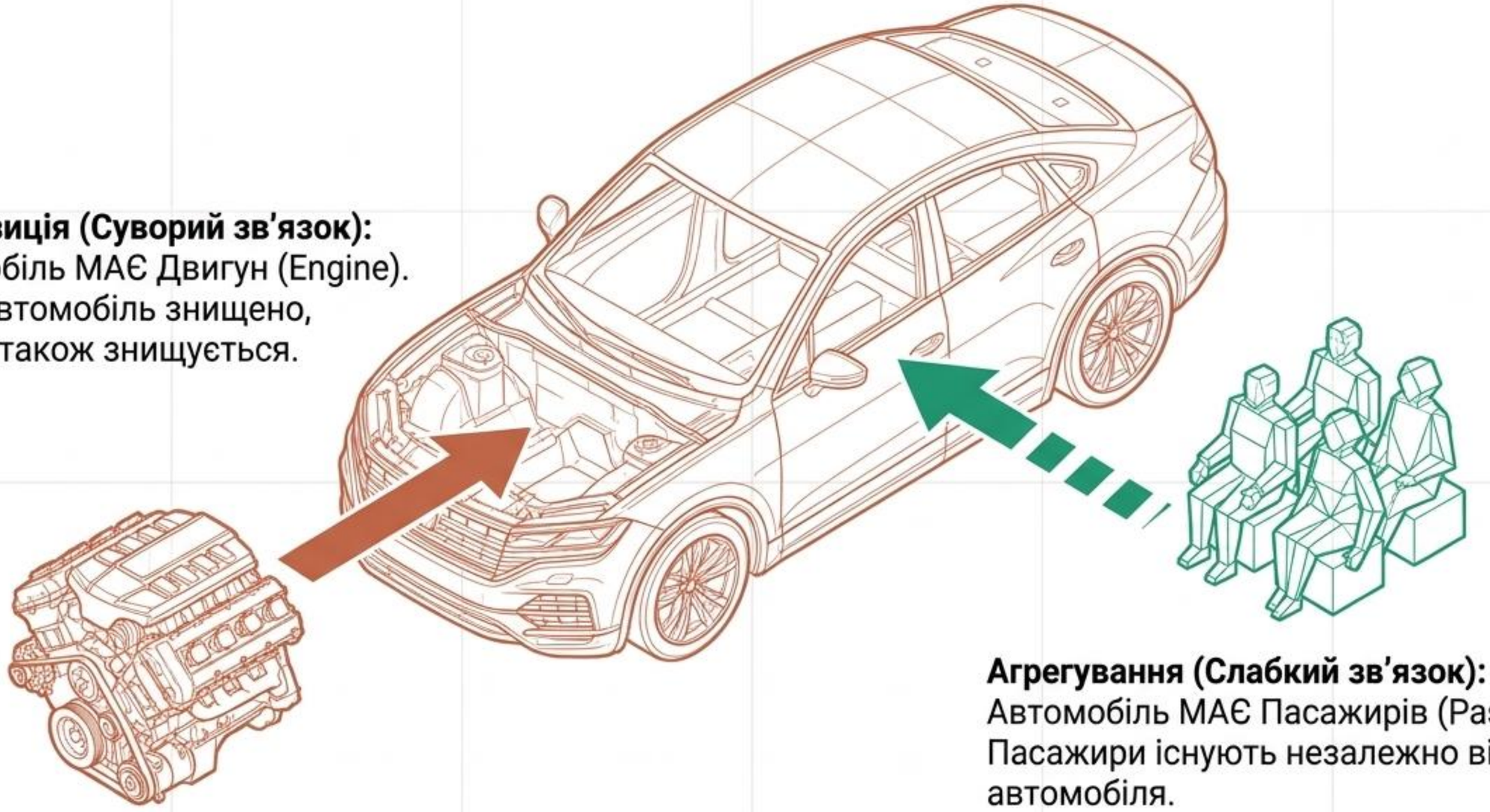
Gemini Notebook



Ідеально **підходить для стабільних систем,** де підкласи строго відповідають контрактам базового класу. Зменшує дублювання коду, але створює жорстку залежність.

Gemini Notebook

Композиція (Суворий зв'язок):
Автомобіль МАЄ Двигун (Engine).
Якщо автомобіль знищено,
двигун також знищується.



Агрегування (Слабкий зв'язок):
Автомобіль МАЄ Пасажирів (Passengers).
Пасажири існують незалежно від
автомобіля.

Ідеально для систем із мінливими вимогами. Поведінка визначається об’єктами, що взаємодіють між собою, а не успадкованим шаблоном.

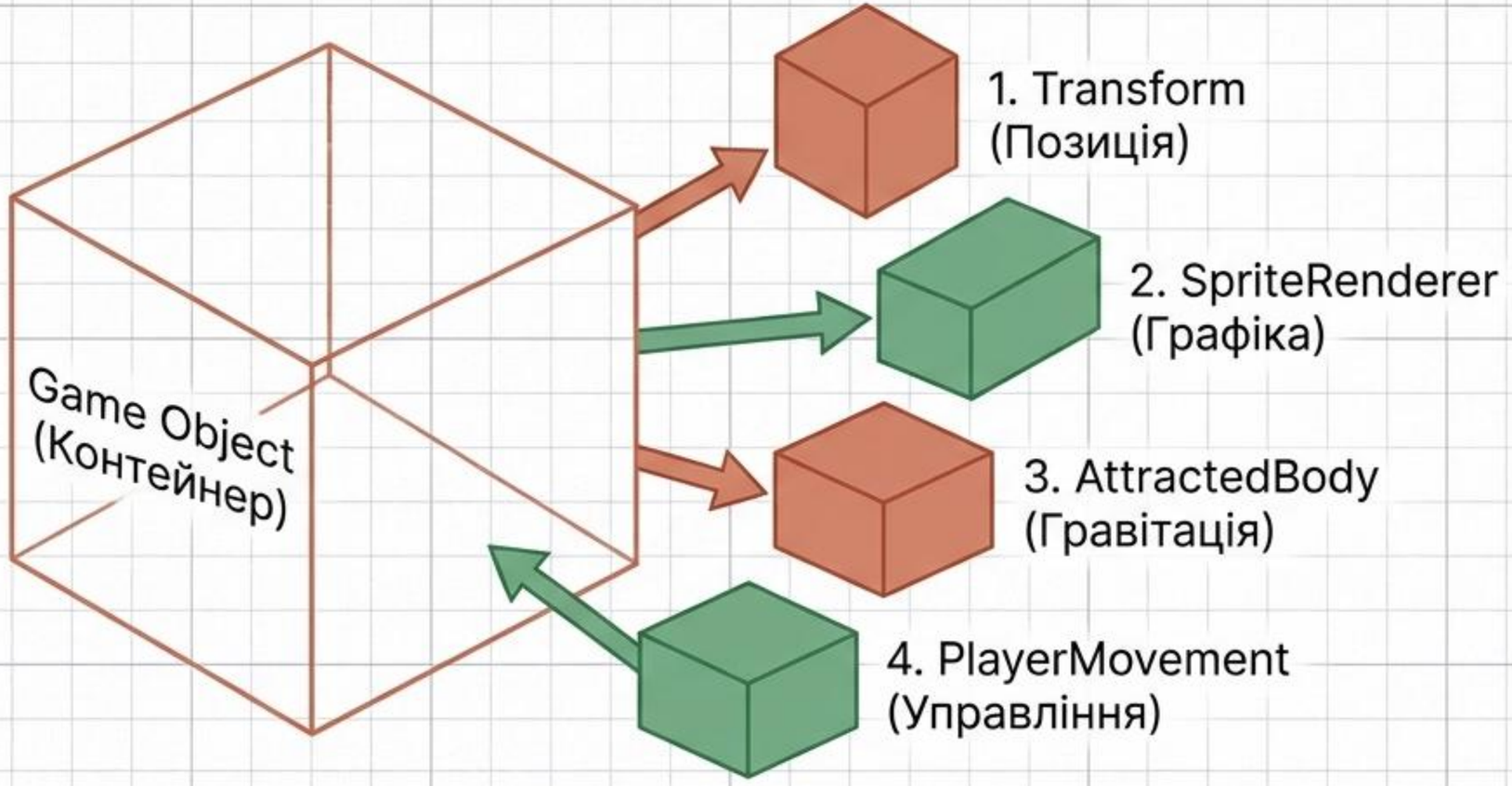
Gemini Notebook

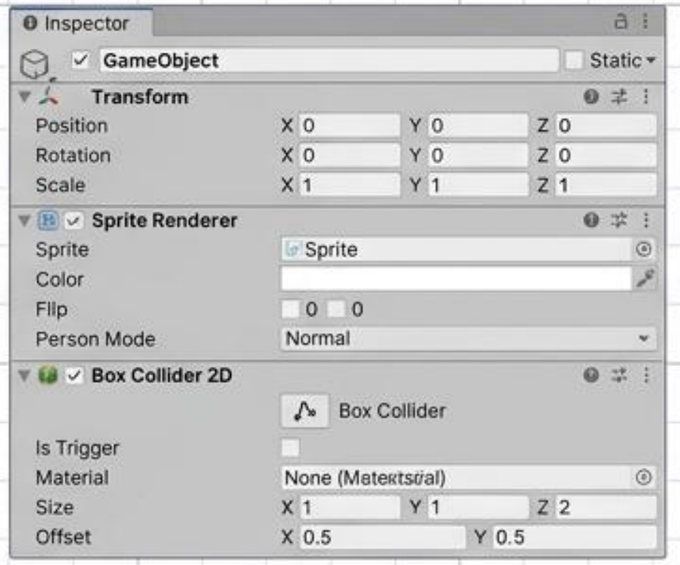
Критерій	Спадкування	Композиція
Тип відношення	«Є» (IS-A)	«Має» (HAS-A)
Гнучкість під час виконання	Статична (визначається під час компіляції)	Динамічна (можна змінювати на льоту)
Залежність коду	Висока (сильна зв'язність, зміни в батьку ламають нащадків)	Низька (слабка зв'язність через інтерфейси)
Ризик «Вибуху класів»	Високий (сотні комбінацій класів)	Низький (перевикористання модулів)

Gemini Notebook

Кейс 1: GameDev та Unity3D (Чому ігри обирають композицію)

Ієрархія класів руйнується в іграх, оскільки сутності мають спільні риси, що перетинаються (фізика є і у гравця, і у каменя, але управління — лише у гравця).





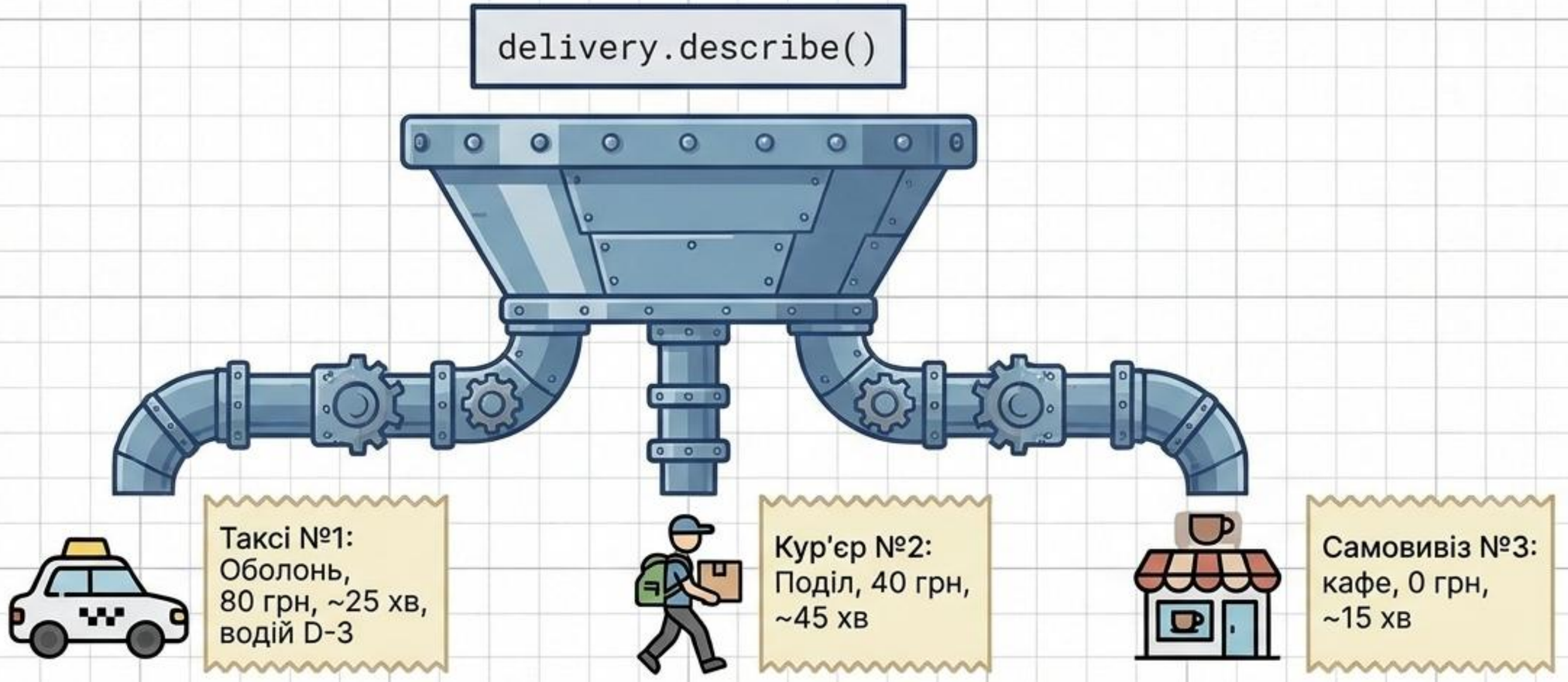

```
public class MyScript : MonoBehaviour
{
    void onMouseDown() {
        var movement = GetComponent<PlayerMovement>();
        var rb = GetComponent<Rigidbody2D>();
        // ... other logic
    }
}
```

GameObject — це лише порожній контейнер.
Функціонал будується виключно завдяки скриптам-компонентам (Композиція).

Gemini Notebook

Кейс 2: Сервіс Доставки (Поліморфізм у дії)

Різні класи реагують на один і той самий метод по-різному. Жодних операторів if для перевірки типу.

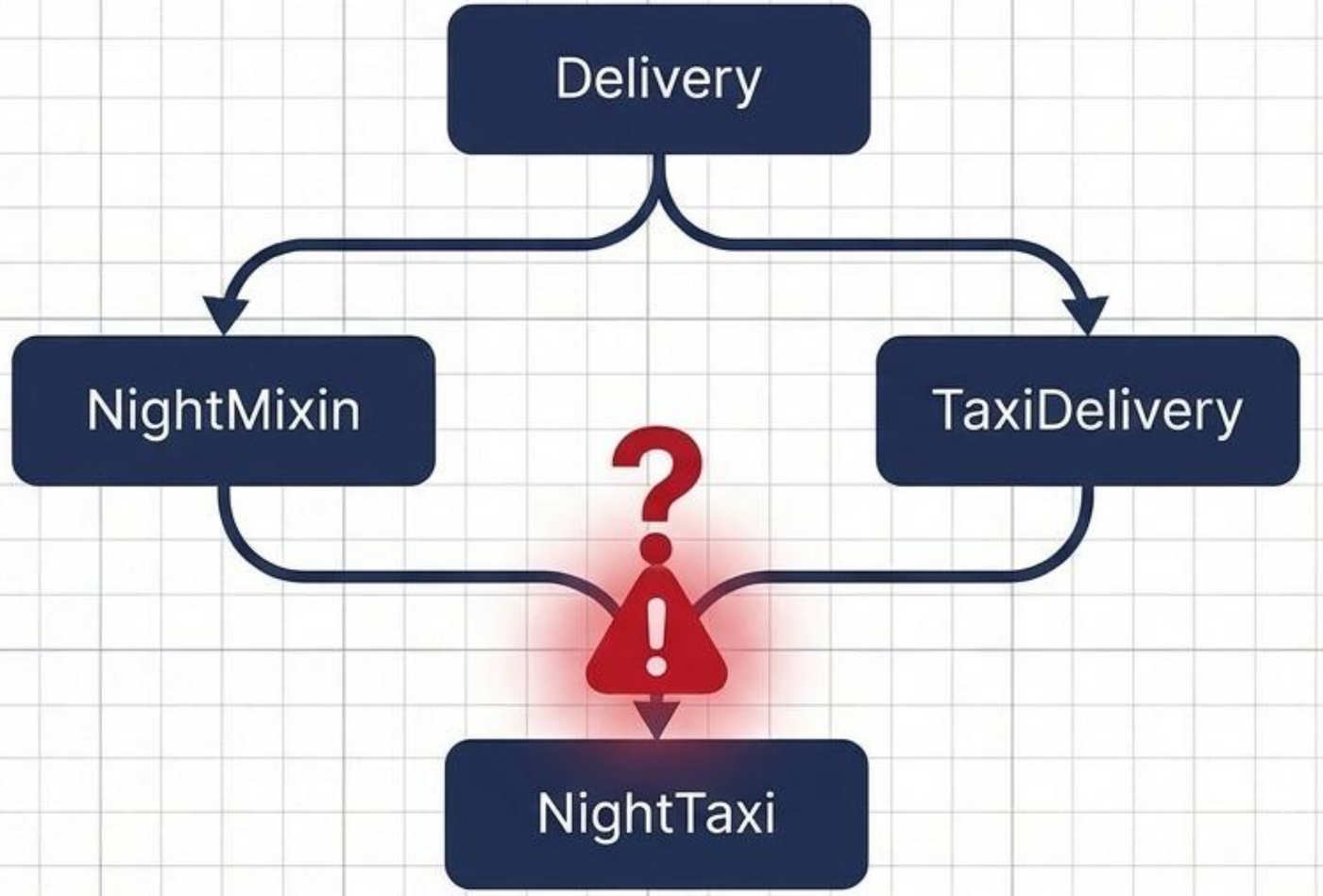


Качина типізація (Duck Typing): Системі байдуже, від кого походить об'єкт, головне — чи має він методи fare() та describe().

Gemini Notebook

Небезпечна зона: Множинне успадкування

Що відбувається, коли клас має кількох батьків? NightTaxi успадковує NightMixin (нічний тариф) та TaxiDelivery (базовий тариф).



Ромбоподібне успадкування (Diamond Problem). Обидва батьки можуть мати метод fare(). Який з них програма викличе першим? Якщо не буде чітких правил, система поводитиметься непередбачувано.

Gemini Notebook

Рішення Python: MRO (Порядок вирішення методів)

MRO — це єдиний, лінійний та передбачуваний список, який вказує програмі точний порядок пошуку методу.



1. NightTaxi
2. NightMixin
3. TaxiDelivery
4. Delivery
5. object

Метод **fare()** буде знайдено спочатку в **NightMixin**.

Gemini Notebook

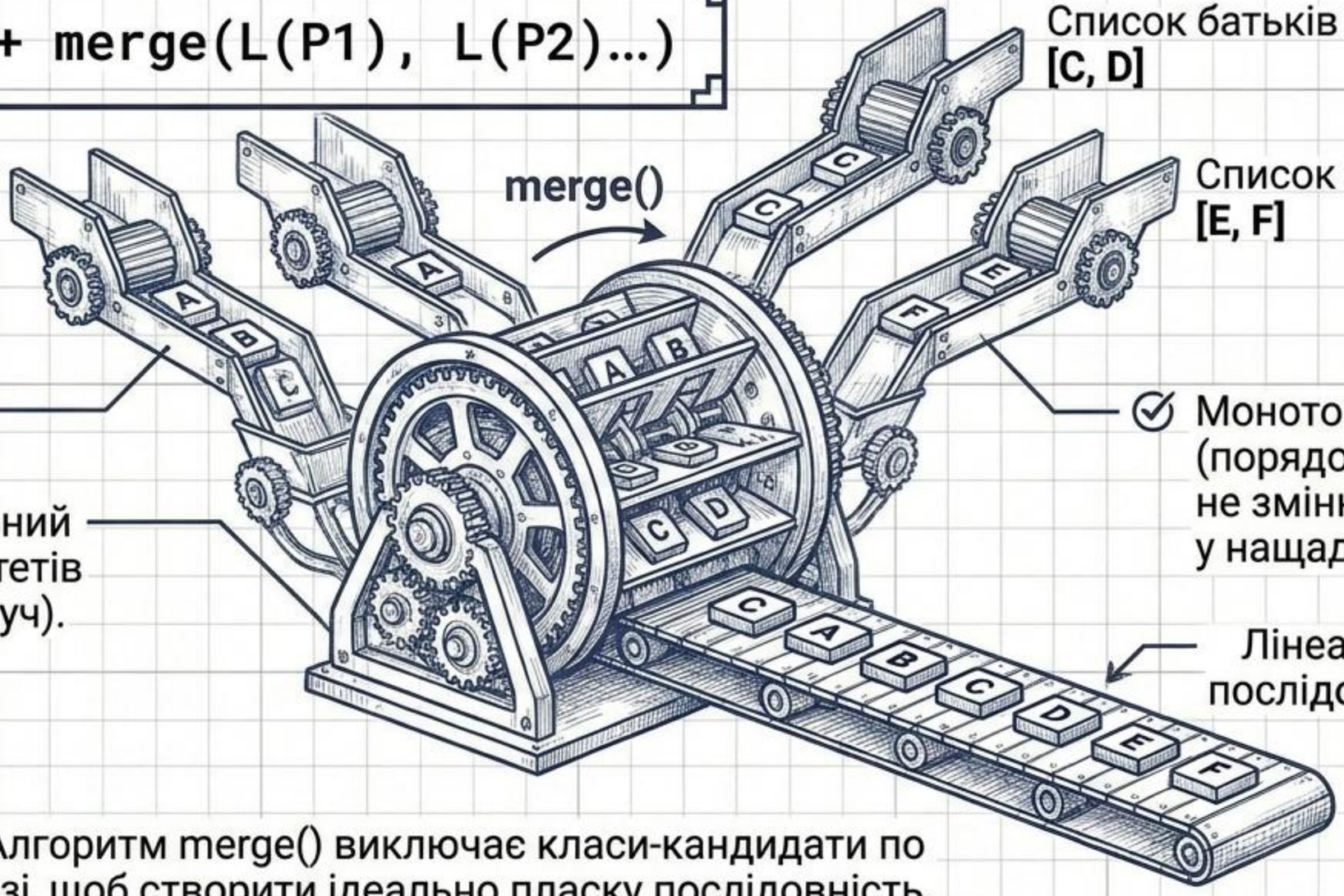
Двигун MRO: Алгоритм C3-Лінеаризації

$$L(C) = [C] + \text{merge}(L(P1), L(P2)\ldots)$$

Список батьків P1
[A, B]

Список батьків P2
[C, D]

Список батьків P3
[E, F]



- Зберігає граф спадкування.
- Зберігає локальний порядок пріоритетів (ліворуч-праворуч).
- Монотонність (порядок предків не змінюється у нащадках).
- Лінеаризована послідовність L(C)

Алгоритм merge() виключає класи-кандидати по черзі, щоб створити ідеально пласку послідовність.

Gemini Notebook

C3-Лінеаризація: Крок за кроком

Алгоритм перевіряє «голову» (перший елемент) кожного списку. Якщо вона не з'являється у «хвості» інших списків, її додають до фінального MRO.

1

$$L(Z) = Z + \text{merge}\begin{pmatrix} FABCO, \\ GDBEO, \\ HDAO, \\ FGH) \end{pmatrix} \ldots$$

2

F

$$Z \text{ extracts } F + \text{merge}\begin{pmatrix} FABCO, \\ GDBEO, \\ HDAO, \\ GH \end{pmatrix} \ldots$$

3

Крок за кроком виключаємо безконфліктні «голови»...

$$L(Z) = Z + \text{merge}\begin{pmatrix} ABCO, \\ GDBEO, \\ HDAO, \\ GH) \end{pmatrix} \ldots)$$

4

Фінальний результат:

Z → F → G → H → D → A → B → C → E → O

Gemini Notebook

Навігація по ланцюжку: Справжня роль super()

super() не означає просто «батьківський клас».
Це означає **«наступний клас у списку MRO»**.

Кооперативне спадкування:
Кожен метод у ланцюжку повинен передати виклик далі **(super().__init__())**.



Якщо хтось «забуде» передати естафетну паличку, класи після нього мовчки не ініціалізуються.

```
super().fare() * 1.3
```

(Передача контролю до наступного класу для застосування нічної надбавки)

Gemini Notebook

Архітектурні Антипатерни (Як робити НЕ треба)



Порушення принципу Лісков (LSP)

Нащадок порушує обіцянку батька. Наприклад, базовий клас повертає число (int), а нащадок раптом повертає рядок («безкоштовно»). Це ламає всю систему.

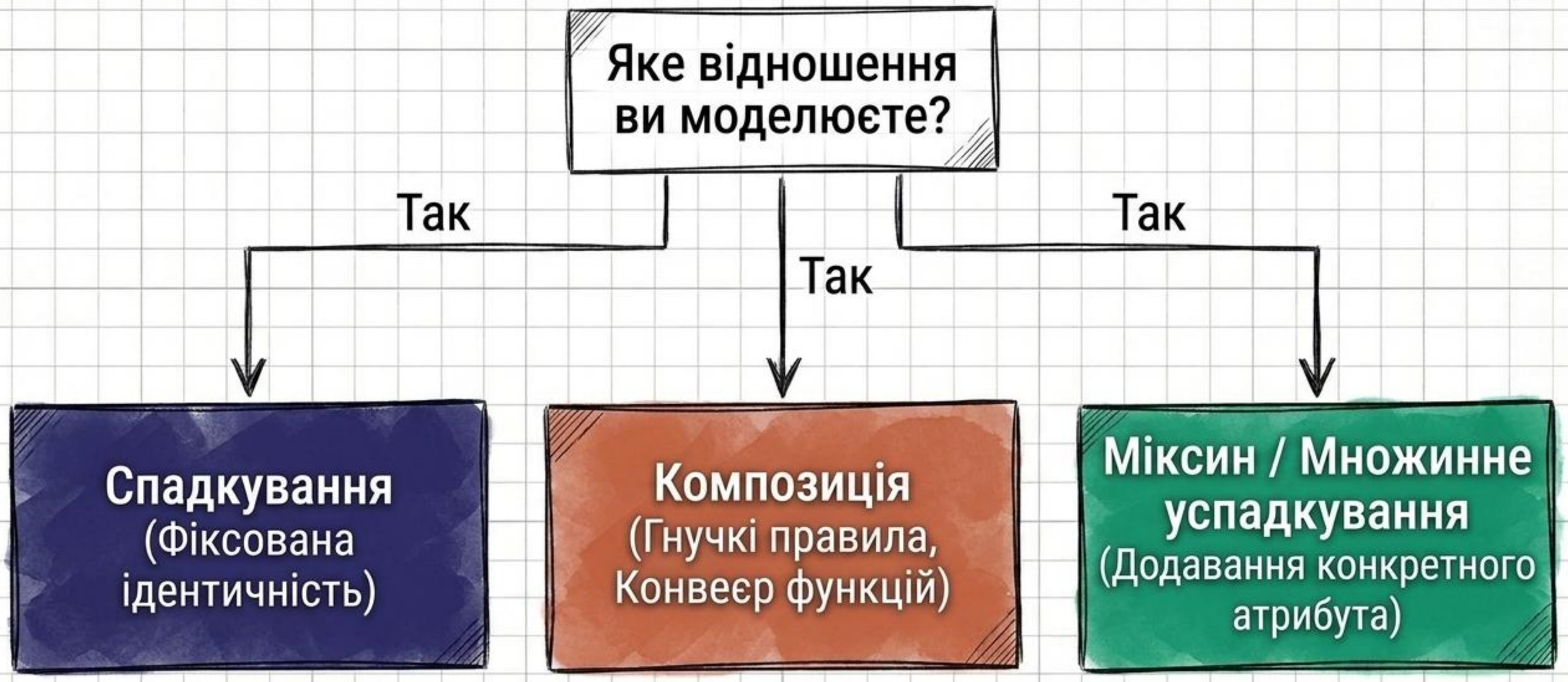


Пастка «Вибуху класів»

Створення десятків класів для кожної дрібної варіації (ТаксіНічПромо, Кур'єрДеньБезПромо). Рішення: використовувати Композицію!

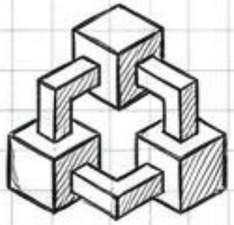
Gemini Notebook

Дерево рішень Архітектора



Gemini Notebook

Ключові принципи Архітектури



Віддавайте перевагу композиції. Використовуйте спадкування лише для стабільних стабільних зв’язків «IS-A».



Поважайте MRO.

Завжди перевіряйте .__mro__, щоб розуміти точний алгоритм C3-лінеаризації.



Естафета super(). Гарантуйте, що кожен метод у ланцюжку янвску множинного успадкування передає виклик далі.



Дотримуйтесь контракту (LSP). Нащадок повинен ідеально замінювати батька без несподіваних змін типів даних чи винятків.

Gemini Notebook